



Banco de Dados

Prof. Elson de Abreu



T-SQL

O que é T-SQL ?

- T-SQL ou Transact-SQL é um dialeto da linguagem SQL (*Structured Query Language*).
- Pode-se dizer que é uma extensão da Microsoft ao padrão SQL-92, acompanhada de melhorias e recursos, assim tornando-a exclusiva do SQL Server.
- É a linguagem primária de acesso a dados utilizada por uma aplicação para acessar e manipular dados e objetos em um banco de dados SQL Server.
- Observe um exemplo em três SGBD's distintos:

SQL Server: `SELECT TOP 10 product, descr, email FROM products`

ORACLE: `SELECT product, descr, email FROM products WHERE ROWNUM <= 10`

MySQL: `SELECT product, descr, email FROM products LIMIT 10`

Mecanismos de controle

- A linguagem T-SQL possui alguns mecanismos de controle, tais como: BEGIN e END, BREAK, CONTINUE, GOTO, IF e ELSE, CASE e ELSE, RETURN, WAITFOR e WHILE.
- Tais mecanismos permitem um maior controle sobre as diversas operações com o SGBD.
- O comando GOTO (controle de fluxo) não será abordado pois cria um desvio incondicional, dificultando a leitura/organização do código, sendo substituído por outros comandos. Já o comando RETURN e CONTINUE pode-se conseguir boas referências no manual da ferramenta.

Estrutura condicional – “if”

- Observe a delimitação do bloco de comandos pelas instruções “begin” e “end”. O comando print foi utilizado apenas para exemplificar uma retorno de valor ao usuário no SQL SERVER.

```
IF MONTH(GETDATE()) = 03
```

```
BEGIN
```

```
    PRINT 'Estamos em Março.'
```

```
END
```

```
ELSE
```

```
BEGIN
```

```
    PRINT 'Não estamos em Março !'
```

```
END
```

Syntax

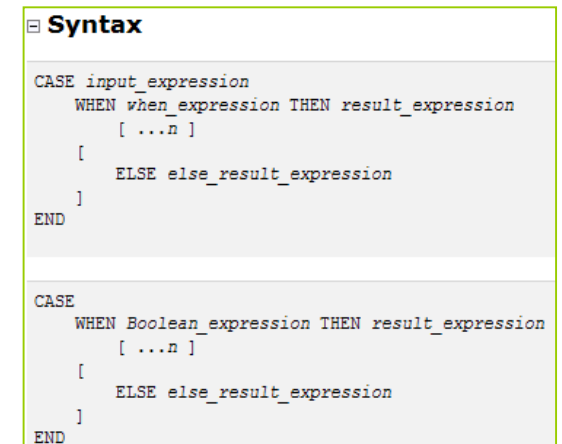
```
IF Boolean_expression { sql_statement | statement_block }  
[  
    ELSE  
    { sql_statement | statement_block } ]
```

(SQL Reference for SQL Server Mobile, 2009)

Estrutura condicional – “case”

- Vale ressaltar que o CASE e o IF não são exatamente a mesma estrutura. É prudente consultar o help da ferramenta.

```
SELECT NOME,  
       TIPOEXTENSO = CASE WHEN TIPOINSTITUICAO = 'PM' THEN 'PUBLICA MUNICIPAL'  
                        WHEN TIPOINSTITUICAO = 'PF' THEN 'PUBLICA FEDERAL'  
                        ELSE 'TIPO-DESCONHECIDO'  
                      END  
FROM INSTITUICOES
```



(SQL Reference for SQL Server Mobile, 2009)

Estrutura de repetição – “while”

- O código abaixo faz exatamente 10 iterações, sendo “quebrada” a repetição abruptamente pelo comando “break”. Observe a condição de repetição WHILE @CONTADOR < 100
- A declaração de variáveis será tratada de forma detalhada posteriormente, por enquanto o que é essencial saber é que todo nome de variável local deve começar pelo símbolo de arroba @.

```
DECLARE @contador INT
SET @contador = 0 -- inicializando a variável

WHILE @contador < 100
BEGIN

    PRINT 'contador = ' + CONVERT(VARCHAR(10), @contador) -- convertendo para string

    SET @contador = @contador + 1

    IF @contador = 10
    BEGIN
        BREAK -- quebra da repetição
    END

END
```

WAITFOR

- O comando WAITFOR permite uma pausa na execução do próximo comando, de no máximo 24hs. Consulte o help da ferramenta para obter maiores detalhes sobre WAITFOR DELAY e WAITFOR TIME (este comando irá aguardar um horário específico).
- O exemplo mostra uma pausa de 1s a cada iteração:

```
DECLARE @contador INT
```

```
SET @contador = 0
```

```
WHILE @contador < 10
```

```
BEGIN
```

```
SET @contador = @contador + 1 -- incremento da variável
```

```
PRINT @contador
```

```
WAITFOR DELAY '00:00:01'
```

```
END
```

Variáveis

- As variáveis do T-SQL se dividem em globais e locais. A variável global é precedida de @@ em seu nome e não pode ser definida pelo usuário, pois a finalidade é retornar informações do sistema. Nas versões mais novas do SQL Server tais variáveis são chamadas de funções.

- Exemplo de variáveis globais:

@@CONNECTIONS

retorna o número de conexões e tentativas desde que o SQL Server iniciou.

@@ERROR

retorna o código do ultimo erro ocorrido

@@IDENTITY

retorna o ultimo valor Identity inserido

@@ROWCOUNT

retorna o número de linhas do último comando executado.

@@VERSION

retorna diversas informações a respeito do servidor.

Exemplo de utilização:

SELECT @@CONNECTIONS ou **PRINT @@CONNECTIONS**

Variáveis locais

- Devem iniciar com @ (arroba).
- Após a criação, o valor default é NULL.
- Sintaxe:

```
DECLARE { @local_variable [AS] data_type, [...n] }
```

- Exemplo:

```
DECLARE @data DATETIME
```

```
DECLARE @idade INT
```

```
DECLARE @nome VARCHAR(25)
```

Exemplos

- **Exemplo 1:** atualizando o CEP dos municípios cujo campo esteja nulo.
- **Exemplo 2:** contagem decrescente utilizando a estrutura WHILE.

```
DECLARE @cep VARCHAR(10)  
SET @cep = '00.000-000'
```

```
UPDATE MUNICIPIOS  
SET CEP = @cep  
WHERE CEP IS NULL
```

```
DECLARE @contador INT  
SET @contador = 10
```

```
WHILE @contador > 0  
BEGIN  
    PRINT 'O contador está com ' + CONVERT(VARCHAR(10),  
        @contador)  
    SET @contador = @contador - 1  
END
```

Exemplos

- Exemplo 3: Atribuindo valores para as variáveis @nome e @login através de uma consulta na tabela USUARIOS.

```
DECLARE @nome varchar(50), @login varchar(50)
```

```
BEGIN
```

```
SELECT @nome = NOME, @login = LOGIN FROM USUARIOS WHERE IDUSUARIO = 100
```

```
PRINT @nome + ' - ' + @login
```

```
END
```

Transações

- Transação é uma sequência de operações que são tratadas como um bloco único e indivisível (atômico).
- Uma transação é delimitada pelos comandos `begin transaction` e `(commit ou rollback)`.
- Se uma transação terminar com `commit` todas as modificações realizadas dentro dela serão efetivadas. Caso termine com `rollback`, as modificações realizadas dentro dela serão desfeitas.

Propriedades de uma Transação

- **Atomicidade:** Ou todas as operações da transação são efetuadas, ou nenhuma delas acontece.
- **Consistência:** Se a transação é consistente e o banco começa consistente, ele termina consistente.
- **Isolamento:** A execução de uma transação é isolada da execução de outras transações.
- **Durabilidade:** Se uma transação é concluída com sucesso (commit bem sucedido), então seus efeitos serão persistentes (duráveis).

Tipos de Transações

- O SQL Server opera com 3 tipos de transações:
 - ✓ Autocommit (modo default)
 - ✓ Explícitas
 - ✓ Implícitas

Transações Autocommit

- Cada comando T-SQL é aplicado ou desfeito quando ele termina (dentro de um mesmo bloco). Se um comando é executado com sucesso ele é aplicado no banco, se o SQL Server encontrar algum erro o comando será desfeito.
- Exemplo de erro de compilação e execução:

```
CREATE TABLE TestBatch (Cola INT PRIMARY KEY, Colb CHAR(3))
```

```
GO
```

```
INSERT INTO TestBatch VALUES (1, 'aaa')
```

```
INSERT INTO TestBatch VALUES (2, 'bbb')
```

```
INSERT INTO TestBatch VALUE (3, 'ccc') /* Syntax error */
```

```
GO
```

```
SELECT * FROM TestBatch /* Returns no rows */
```

```
GO
```

❖ *Nenhum dos comandos INSERT será aplicado devido ao erro de compilação. Isso ocorre devido ao bloco delimitado pelo comando GO.*

Transações Autocommit

- Cada comando T-SQL é aplicado ou desfeito quando ele termina (dentro de um mesmo bloco). Se um comando é executado com sucesso ele é aplicado no banco, se o SQL Server encontrar algum erro o comando será desfeito.
- Exemplo de erro de compilação e execução:

```
CREATE TABLE TestBatch (Cola INT PRIMARY KEY, Colb CHAR(3))
```

```
GO
```

```
INSERT INTO TestBatch VALUES (1, 'aaa')
```

```
INSERT INTO TestBatch VALUES (2, 'bbb')
```

```
INSERT INTO TestBatch VALUE (3, 'ccc') /* Syntax error */
```

```
GO
```

```
SELECT * FROM TestBatch /* Returns no rows */
```

```
GO
```

❖ *Nenhum dos comandos INSERT será aplicado devido ao erro de compilação. Isso ocorre devido ao bloco delimitado pelo comando GO.*

Transações Explícitas

- É uma transação na qual você define explicitamente o seu início e seu final. Transações explícitas duram somente até a transação terminar. Quando a transação termina, a conexão retorna ao modo de transação que estava antes que a transação explícita começou.
- Exemplo utilizando o BD “Formula1”:

BEGIN TRAN

```
INSERT INTO PILOTO (PILOTO_ID, NOME, PAIS_ORIGEM, PONTOS, EQUIPE_ID)
VALUES (7, 'Nelson Piquet', 'Brasil', NULL, 1)
```

```
UPDATE PILOTO
SET PONTOS = 0
WHERE PONTOS IS NULL
```

COMMIT -- A transação poderia terminar em ROLLBACK, caso deseja-se desfazer os comandos acima.

Exemplo

Exemplo utilizando o BD “Formula1”: inserção de um novo piloto com chave primária duplicada

(PILOTO_ID = 6), irá produzir um erro e será tratado.

```
DECLARE @NUMERRO INT
```

```
BEGIN TRAN
```

```
INSERT INTO PILOTO (PILOTO_ID, NOME, PAIS_ORIGEM, PONTOS, EQUIPE_ID)
```

```
VALUES (6, 'Nelson Piquet', 'Brasil', NULL, 1)
```

```
SET @NUMERRO = @@ERROR
```

```
IF @NUMERRO > 0
```

```
BEGIN
```

```
ROLLBACK TRAN
```

```
PRINT 'FALHA AO TENTAR INSERIR NOVO PILOTO'
```

```
END
```

```
ELSE
```

```
BEGIN
```

```
COMMIT TRAN
```

```
END
```

❖ Consulte o help da ferramenta para obter maiores informações sobre transações com nomes, save transaction etc. O código de erro igual a zero quer dizer que todas as operações ocorreram com sucesso.

Transações Implícitas

- Depois que um modo de transação implícita é ativado para uma conexão, o SQL Server automaticamente inicia a transação quando ele executa um dos seguintes comandos:

ALTER TABLE;
DELETE;
FETCH;

INSERT;
REVOKE;
TRUNCATE TABLE;

CREATE;
DROP;
GRANT;

OPEN;
SELECT;
UPDATE;

- A transação fica pendente até que seja emitido um comando COMMIT ou ROLLBACK. Depois que a primeira transação for aplicada ou desfeita, o SQL Server automaticamente inicia uma nova transação da próxima vez que um destes comandos for executado pela conexão.
- O modo de transação implícita é ativado utilizando o comando T-SQL *SET IMPLICIT_TRANSACTIONS ON*.

Tal modo é utilizado até que a conexão execute um comando *SET IMPLICIT_TRANSACTIONS OFF*, que irá retornar a conexão ao modo autocommit.

Exemplo

```
CREATE TABLE ImplicitTran (Cola int PRIMARY KEY, Colb char(3) NOT NULL)
GO
SET IMPLICIT_TRANSACTIONS ON
GO
INSERT INTO ImplicitTran VALUES (1, 'aaa') --Primeira transação implícita iniciada
GO
INSERT INTO ImplicitTran VALUES (2, 'bbb')
GO
COMMIT TRANSACTION --Primeira transação implícita aplicada
GO
SELECT COUNT(*) FROM ImplicitTran --Segunda transação implícita iniciada
GO
INSERT INTO ImplicitTran VALUES (3, 'ccc')
GO
SELECT * FROM ImplicitTran
GO
COMMIT TRANSACTION --Segunda transação implícita aplicada
GO
SET IMPLICIT_TRANSACTIONS OFF
GO
```

Tratamento de Erros

- Os erros normalmente eram tratados verificando o valor da variável global @@ERROR, que guarda o código de erro da última instrução T-SQL executada.
- A partir da versão 2005 existe um mecanismo de tratamento de erros que consiste na seguinte sintaxe:

BEGIN TRY

...

END TRY

BEGIN CATCH

...

END CATCH

} Captura a exceção

Exemplo

Exemplo: tentando inserir um registro com PK duplicada (BD “Formula1”).

```
BEGIN TRY
```

```
    BEGIN TRAN
```

```
        INSERT INTO PILOTO (PILOTO_ID, NOME, PAIS_ORIGEM, PONTOS, EQUIPE_ID)
            VALUES (6, 'Nelson Piquet', 'Brasil', NULL, 1)
```

```
    COMMIT TRAN
```

```
END TRY
```

```
BEGIN CATCH
```

```
    ROLLBACK TRAN
```

```
    SELECT ERROR_NUMBER() AS NumeroErro, ERROR_SEVERITY() AS Severidade, ERROR_STATE() as Estado, ERROR_MESSAGE() as Mensagem
```

```
END CATCH
```